Übung 4

JavaScript (JS)

Einführung in Webengineering · WS 2026/27

In acht Aufgaben untersuchen Sie JavaScript und schreiben eigene Funktionen und Objekte. Begründen Sie bei Codeanalysen zuerst Ihre Vermutung und prüfen Sie diese danach im Browser.

01 JavaScript ausführen und Typen untersuchen

a) Legen Sie im Ordner `kapitel04` eine `index.html` und eine `aufgabe01.js` an. Binden Sie die JavaScript-Datei mit `<script src="aufgabe01.js"></script>` am Ende von `body` ein. Geben Sie eine Begrüßung mit `console.log()` aus und öffnen Sie die Konsole der Entwicklerwerkzeuge.

b) Bestimmen und prüfen Sie die folgenden Ausgaben. Erklären Sie die Besonderheiten bei `null`, Arrays und Funktionen. Warum ist `Array.isArray()` für die Erkennung eines Arrays hilfreich?

```
console.log(typeof 7, typeof "7", typeof true);
console.log(typeof undefined, typeof null);
console.log(typeof [7], typeof { kurs: "JS" });
console.log(typeof function () {}, Array.isArray([7]));
```

Weisen Sie anschließend derselben Variablen nacheinander `7`, `"sieben"` und `[7]` zu. Prüfen Sie jeweils den Typ und erklären Sie daran dynamische Typisierung.

c) Prüfen und erläutern Sie die Ergebnisse von `0.1 + 0.2 === 0.3`, `1 / 0`, `Number("Web")` und `typeof NaN`. Was zeigen diese Beispiele über den Zahlentyp `Number`?

02 Typumwandlungen und Vergleiche verstehen

a) Bestimmen Sie die Ausgaben und die Datentypen der ersten drei Ergebnisse. Zeigen Sie, wo eine explizite oder implizite Typumwandlung erfolgt und wo ohne Typumwandlung verglichen wird.

```
console.log("5" + 2, "5" - 2, Number("5") + 2);
console.log(0 == "0", 0 === "0");
console.log(null == undefined, null === undefined);
```

b) Ordnen Sie `0`, `"0"`, `""`, `" "`, `[]` und `{}` als truthy oder falsy ein und prüfen Sie Ihre Einschätzung mit `Boolean()`. Erklären Sie den Unterschied zwischen einem leeren String und einem String mit einem Leerzeichen. Weshalb sollte man für Gleichheitsvergleiche in der Regel `===` verwenden?

Funktionen und Sichtbarkeit untersuchen

03 Parameter und Funktionen als Werte verwenden

a) Die folgende Funktion soll einen Preis nach Abzug eines prozentualen Rabatts liefern. Ohne zweites Argument sollen `10%` gelten; ein ausdrücklich angegebener Rabatt von `0%` muss erhalten bleiben.

```
function preis(betrag, rabatt) {
  rabatt = rabatt || 10;
  return betrag * (1 - rabatt / 100);
}
```

Prüfen Sie `preis(100)`, `preis(100, 0)` und `preis(100, 20)` und erklären Sie den Fehler. Korrigieren Sie die Funktion durch eine Prüfung auf `undefined`; erwartet werden `90`, `100` und `80`.

b) Schreiben Sie `zweimal(f, x)`: Die Funktion soll `f` auf `x` anwenden und danach `f` auf das Zwischenergebnis anwenden. Geben Sie das Endergebnis zurück. Testen Sie mit einer Funktion, die `5` addiert, und einer Funktion, die quadriert: Aus `10` soll im ersten Fall `20`, aus `3` im zweiten Fall `81` werden. Erklären Sie, warum Sie dabei `f` statt eines bereits berechneten Ergebnisses übergeben.

04 Scope und Hoisting nachvollziehen und Fehler finden

a) Welche drei Werte werden ausgegeben? Erläutern Sie den Gültigkeitsbereich der beiden Variablen und unterscheiden Sie Deklaration und Zuweisung. Prüfen Sie mit einem Haltepunkt vor der ersten Ausgabe den Wert der lokalen Variablen.

```
var wert = "außen";
function pruefen() {
  console.log(wert);
  var wert = "innen";
  console.log(wert);
}
pruefen();
console.log(wert);
```

b) Führen Sie die beiden Versuche getrennt in einer frisch geladenen Seite aus. Welcher Aufruf funktioniert, welcher löst welchen Fehlertyp aus? Erklären Sie den Unterschied zwischen Funktionsdeklaration und Funktionsausdruck.

```
// Versuch 1
start();
function start() { console.log("Start"); }
```

```
// Versuch 2
start();
var start = function () { console.log("Start"); };
```

c) Welchen Wert liefert `daten()`? Begründen Sie dies mit der automatischen Semikolon-Einfügung und ändern Sie den Code so, dass das Objekt zurückgegeben wird.

```
function daten() {
  return
  { kurs: "JS" };
}
```

Objekte darstellen und Daten kapseln

05 Objekte teilen und von JSON unterscheiden

a) Bestimmen Sie die Ausgaben. Unterscheiden Sie die Änderung einer Objekteigenschaft von der Zuweisung eines neuen Objekts an einen Parameter. Entsteht durch `var kopie = kurs` eine unabhängige Kopie?

```
var kurs = { titel: "JavaScript", raum: "A101" };
var kopie = kurs;
function verschieben(obj) {
  obj.raum = "B202";
  obj = { titel: "CSS", raum: "C303" };
}
verschieben(kopie);
console.log(kurs.raum, kopie === kurs);
```

b) Ergänzen Sie am Objekt `kurs` ein Array `termine` mit `"Montag"` und `"Mittwoch"` sowie eine Methode `beschreibung()`. Diese soll den Titel und den aktuellen Raum als zusammenhängenden String zurückgeben. Prüfen Sie die Methode vor und nach einer weiteren Raumänderung.

c) Stellen Sie die Daten `titel`, `raum` und `termine` zusätzlich als gültiges JSON dar. Erläutern Sie die Regeln für Eigenschaftsnamen und Strings sowie den Unterschied zwischen einer Methode im JavaScript-Objekt und den in JSON darstellbaren Daten.

06 Objekte mit Fabrikfunktionen kapseln

a) Schreiben Sie die Fabrikfunktion `erzeugeKurs(titel)`. Sie soll ein neues Objekt mit der Methode `beschreibung()` zurückgeben. Die Methode liefert `"Kurs: "` gefolgt vom übergebenen Titel. Der Titel bleibt ausschließlich im Funktionsparameter gespeichert und wird nicht als öffentliche Objekteigenschaft angelegt. Verwenden Sie für die Methode die `function`-Schreibweise.

b) Testen Sie Ihre Funktion mit zwei unabhängigen Kursobjekten:

```
var js = erzeugeKurs("JavaScript");
var css = erzeugeKurs("CSS");
console.log(js.beschreibung());
console.log(css.beschreibung());
console.log(js.titel);
js.titel = "HTML";
console.log(js.beschreibung());
```

Die ersten beiden Beschreibungen sollen `"Kurs: JavaScript"` und `"Kurs: CSS"` lauten. Erklären Sie die beiden letzten Ausgaben. Warum kann die Methode weiterhin auf den Parameter zugreifen, obwohl der Aufruf von `erzeugeKurs()` bereits beendet ist? Unterscheiden Sie den Parameter `titel` von der später angelegten Eigenschaft `js.titel`.

Aufrufe und Prototypen verstehen

07 Dynamische Bindung und Klickereignisse verstehen

a) Bestimmen Sie die Ausgaben und geben Sie für jeden Methodenaufruf an, worauf `this` verweist. Warum kann dieselbe Funktion unterschiedliche Ergebnisse liefern?

```
var anna = {
  name: "Anna",
  vorstellen: function () { return this.name; }
};
var ben = { name: "Ben", vorstellen: anna.vorstellen };
console.log(anna.vorstellen(), ben.vorstellen());
```

b) Erstellen Sie eine kleine HTML-Seite mit zwei Schaltflächen nach dem folgenden Muster. Ergänzen Sie die Funktion `markieren(element)`, die den Hintergrund der übergebenen Schaltfläche mit `element.style.backgroundColor = "gold"` einfärbt.

```
<button type="button" onclick="markieren(this)">Markieren</button>
```

Prüfen Sie, dass jeder Klick nur die angeklickte Schaltfläche verändert. Erklären Sie, worauf `this` im Attribut `onclick` verweist und welcher Wert dadurch an den Parameter `element` übergeben wird.

08 Eigenschaften entlang der Prototypenkette finden

Gegeben ist eine Kette aus drei selbst angelegten Objekten:

```
var medium = { art: "Lernmaterial" };
var buch = Object.create(medium);
buch.beschreibung = function () {
  return this.titel + " (" + this.art + ")";
};
var jsBuch = Object.create(buch);
jsBuch.titel = "JavaScript";
```

a) Zeichnen Sie die Prototypenkette ausgehend von `jsBuch` bis zu `null`. Geben Sie an, auf welchem Objekt `titel`, `beschreibung` und `art` jeweils gefunden werden. Prüfen Sie die Kette mit `Object.getPrototypeOf()` und bestimmen Sie das Ergebnis von `jsBuch.beschreibung()`.

b) Erzeugen Sie mit `Object.create(buch)` ein zweites Objekt `cssBuch` mit dem eigenen Titel `"CSS"`. Prüfen und erklären Sie `jsBuch.beschreibung === cssBuch.beschreibung`. Welchen Vorteil hat die gemeinsam verwendete Methode?

c) Setzen Sie `jsBuch.art = "E-Book"`. Welche Werte liefern nun `jsBuch.art`, `cssBuch.art` und `medium.art`? Führen Sie danach `delete jsBuch.art` aus und prüfen Sie erneut `jsBuch.art`. Erklären Sie das Verdecken einer geerbten Eigenschaft durch eine eigene Eigenschaft.